

Test e Strategia di Qualità

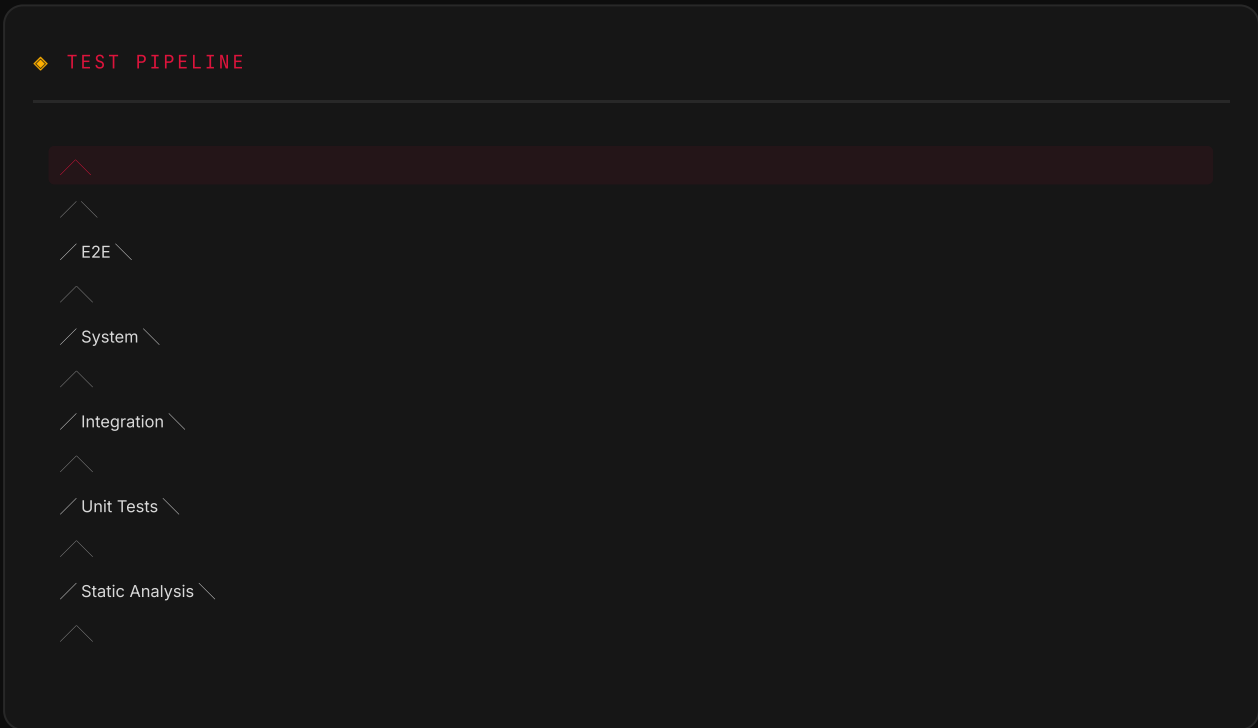
Unit Test, Integration Test, E2E e CI/CD Pipeline

- Unit Test, Integration Test, E2E e CI/CD Pipeline

Categoria: Qualità del Software Copertura Target: 80% C++ core · 70% React components

- 1. Panoramica della Strategia di Test

WhyCremisi adotta la **testing pyramid** classica, adattata a un progetto ibrido C++/React con componenti audio in tempo reale.



LIVELLO	TECNOLOGIA	TARGET COPERTURA	ESECUZIONE
Unit (C++)	JUCE Unit Test Framework	80% core engine	<code>make test</code> / CTest
Unit (React)	Jest + RTL	70% components	<code>npm run test</code>
Integration	Mock DAW Server + MSW	60% API paths	CI (every push)
System	Headless Plugin Host	50% workflows	Nightly
E2E	Playwright	40% user flows	CI (main branch)

[NOTE] I test audio in tempo reale richiedono un mock engine dedicato. I test di latenza non possono essere eseguiti in ambienti CI standard senza hardware audio dedicato.

• 2. Test C++

— FRAMEWORK

Utilizziamo il **JUCE Unit Test Framework** nativo, attivato tramite il flag di compilazione `AUDIO_PROCESSOR_UNIT_TESTS`. I test sono integrati nel modulo `WhyCremisi_Tests` e compilati come target separato.

```
// Esempio: test di un messaggio OSC per il bridge
#ifdef AUDIO_PROCESSOR_UNIT_TESTS
class OscBridgeTest : public UnitTest
{
public:
    OscBridgeTest() : UnitTest ("OSC Bridge", "Messaging") {}

    void runTest() override
    {
        beginTest ("Parameter change message serialization");
        {
            auto msg = OscMessage ("/param/set")
                .withFloat (0.5f);
            expectEquals (msg.getFloat(), 0.5f);
        }
        beginTest ("DAW discovery heartbeat");
        {
            auto hb = OscMessage ("/daw/heartbeat")
                .withInt (12345);
            expectEquals (hb.getInt(), 12345);
        }
    }
};

static OscBridgeTest oscBridgeTest;
#endif
```

— CATEGORIE DITEST

CATEGORIA	DESCRIZIONE	QUANTITÀ
OscBridge Messaging	Serializzazione/deserializzazione messaggi OSC, heartbeat DAW, routing errori	~45 test
PluginProcessor Parameters	Get/set parametri, preset load/save, stato bypass, null safety	~60 test
DawDetector OSC Parsing	Riconoscimento DAW (Ableton/Reaper/Logic/Cubase/FL), heartbeat timeout, reconnect	~35 test
Audio Analysis	FFT processing, LUFS measurement, peak detection, spectral centroid	~50 test
Memory Management	RAll checks, leak detection, shared pointer validity	~25 test

— MOCK DAW SERVER PER TEST DI INTEGRAZIONE

Un server DAW mockato in C++ simula i messaggi OSC di una DAW reale. I test di integrazione verificano:

1. Invio heartbeat periodico → risposta del bridge
2. Modifica parametro → aggiornamento UI via WebSocket
3. Disconnessione improvvisa → riconnessione automatica
4. Caricamento preset → verifica stato parametri

— MEMORY LEAK DETECTION

Compiliamo con **AddressSanitizer (ASAN)** su macOS e Linux, e **Dr. Memory** su Windows:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug \  
-DWHYCREMISI_ENABLE_ASAN=ON \  
-DWHYCREMISI_ENABLE_UNIT_TESTS=ON
```

[NOTE] ASAN è disabilitato nei build Release e nel plugin caricato in DAW (l'overhead di memoria è incompatibile con le performance audio in tempo reale).

• 3. Test React

— FRAMEWORK

Utilizziamo **Jest + React Testing Library** per i componenti frontend. I test sono eseguiti con `jsdom` come environment e includono un **Mock Service Worker (MSW)** per simulare il server WebSocket.

```
npm test
✓ renders BoxChat with message history
✓ renders Oscilloscope with mock audio data
✓ renders CorrelationMeter with stereo signal
✓ DAW track list updates on state change
✓ WebSocket reconnection after disconnect
✓ BoxResize responds to drag events
✓ SetupScreen AI provider selection
```

CATEGORIE DI TEST

CATEGORIA	DESCRIZIONE	QUANTITÀ
Box Components	BoxChat, BoxSearch, BoxInfo, BoxResize, BoxSetup — render corretto e interazioni	~30 test
WebSocket Connection	Connect/disconnect/reconnect, message parsing, error states, timeout	~20 test
Oscilloscope & Correlation	Render canvas, dati FFT mockati, aggiornamento in tempo reale, resize	~15 test
DAW Track List	Stato tracce, selezione, routing, muting/solo, aggiornamenti OSC	~20 test
Session Panel	FlightRecorder playback, snapshot restore, history navigation	~15 test

MOCK WEBSOCKET CON MSW

```
import { setupServer } from 'msw/node'
import { handlers } from './mocks/handlers'

const server = setupServer(...handlers)

beforeAll(() => server.listen())
afterEach(() => server.resetHandlers())
afterAll(() => server.close())

it('should reconnect after WebSocket disconnect', async () => {
  const { result } = renderHook(() => useWebSocket())

  act(() => { result.current.disconnect() })
  await waitFor(() => expect(result.current.status).toBe('disconnected'))

  act(() => { result.current.reconnect() })
  await waitFor(() => expect(result.current.status).toBe('connected'))
})
```

[NOTE] MSW intercetta le richieste WebSocket a livello di network, permettendo test deterministici senza un server reale. Fondamentale per i test in CI dove non è disponibile il backend C++.

• 4. Test End-to-End

— FRAMEWORK

Utilizziamo **Playwright** per i test E2E. Il C++ plugin bridge è eseguito in modalità standalone headless, mentre il frontend React viene servito da Vite in modalità preview.

◆ OVERVIEW

e2e/

startup.spec.ts # Avvio → connessione DAW → UI pronta

parameter-tweak.spec.ts # Modifica parametro → OSC update → UI sync

preset-load.spec.ts # Carica preset → verifica parametri

ai-conversation.spec.ts # Chat AI → risposta → esecuzione azione

reconnect.spec.ts # Disconnessione → riconnessione → stato restored

— FLUSSI TESTATI

FLUSSO	PASSAGGI	ASSERTION
Startup	Avvia standalone → carica UI → connessione WebSocket → heartbeat DAW	BotFace visibile, connection status "connected"
Parameter Tweak	Muovi slider parametro → messaggio OSC al bridge → update UI	Valore parametro aggiornato in < 50ms
Preset Load	Seleziona preset → bridge carica parametri → UI si aggiorna	Tutti i parametri corrispondono al preset
AI Conversation	Invia messaggio → AI risponde → advisory card → Execute/Dismiss	Card appare, azione eseguita o annullata
Reconnection	Kill bridge process → UI mostra "disconnected" → restart bridge → reconnect	Stato ritorna "connected", sessione restored

— CONFIGURAZIONE HEADLESS CI

```
# playwright.config.ts
export default defineConfig({
  testDir: './e2e',
  fullyParallel: true,
  retries: 2,
  workers: process.env.CI ? 2 : undefined,
  use: {
    baseURL: 'http://localhost:5173',
    headless: true,
  },
  webServer: {
    command: 'npm run build && npm run preview',
    port: 5173,
    reuseExistingServer: !process.env.CI,
  },
})
```


— WORKFLOW GITHUB ACTIONS

```
name: WhyCremisi CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-cpp:
    strategy:
      matrix:
        os: [macos-14, windows-2022, ubuntu-22.04]
    runs-on: ${ matrix.os }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/cache@v4
        with:
          path: ~/.ccache
          key: ccache-${ matrix.os }-${ hashFiles('**/CMakeLists.txt') }}
      - run: |
          cmake -S . -B build \
            -DCMAKE_BUILD_TYPE=Release \
            -DWHYCREMISI_ENABLE_UNIT_TESTS=OFF
          cmake --build build --target WhyCremisi_Standalone
      - uses: actions/upload-artifact@v4
        with:
          name: whycremisi-${ matrix.os }
          path: build/WhyCremisi_artefacts/

  test-cpp:
    runs-on: macos-14
    steps:
      - uses: actions/checkout@v4
      - run: |
          cmake -S . -B build \
            -DCMAKE_BUILD_TYPE=Debug \
            -DWHYCREMISI_ENABLE_UNIT_TESTS=ON \
            -DWHYCREMISI_ENABLE_ASAN=ON
          cmake --build build --target WhyCremisi_Tests
          ctest --test-dir build --output-on-failure

  test-frontend:
    runs-on: ubuntu-22.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm run lint
      - run: npm run test -- --coverage
      - uses: codecov/codecov-action@v4
        with:
          directory: coverage
```

```
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: 20
  - run: npm ci
  - run: npx playwright install
  - run: npm run build
  - run: npm run test:e2e
```

— PIPELINE A COLPO D'OCCHIO

◆ TEST PIPELINE

Commit

→ → →

Build Test Lint +

C++ C++ Test FE

E2E Test

Package

Deploy

— ARTEFATTI PRODOTTI

PIATTAFORMA	FORMATO	PATH
macOS	WhyCremisi.vst3	build/WhyCremisi_artefacts/Release/VST3/
macOS	WhyCremisi.component	build/WhyCremisi_artefacts/Release/AU/
macOS	WhyCremisi.app	build/WhyCremisi_artefacts/Release/Standalone/
Windows	WhyCremisi.vst3	build/WhyCremisi_artefacts/Release/VST3/
Windows	WhyCremisi.exe	build/WhyCremisi_artefacts/Release/Standalone/
Linux	WhyCremisi.vst3	build/WhyCremisi_artefacts/Release/VST3/

— SMOKE TEST CHECKLIST (PRE-RELEASE)

◆ OVERVIEW

- Plugin si carica in Ableton Live 11/12 senza crash
- Plugin si carica in Reaper 7 senza crash
- Plugin si carica in Logic Pro senza crash
- Plugin si carica in Cubase 12/13 senza crash
- Plugin si carica in FL Studio 21 senza crash
- Standalone si avvia e mostra interfaccia correttamente
- WebSocket bridge si connette entro 2 secondi
- BotFace animata appare nello stato "idle"
- BoxChat accetta input e mostra risposta AI
- AI provider locale (Ollama) funziona offline
- AI provider cloud (Gemini/OpenAI) funziona online
- Parametri plugin si aggiornano in tempo reale
- Preset load non altera lo stato audio in modo errato
- Disconnessione DAW → UI mostra stato corretto → riconnessione
- Session restore dopo crash
- Flight Recorder cattura e riproduce snapshots

— MATRICE DI COMPATIBILITÀ DAW

DAW	VERSIONE	MACOS	WINDOWS	LINUX	NOTE
Ableton Live	11, 12	✓ Full	✓ Full	—	Testato con VST3 e AU
Reaper	7	✓ Full	✓ Full	✓ Beta	OSC nativo ottimale
Logic Pro	10.8+	✓ Full	—	—	Solo AU, test priority
Cubase	12, 13	✓ Full	✓ Full	—	VST3, test automazioni
FL Studio	21	—	✓ Full	—	Windows only, VST3
Pro Tools	2024	△ Parz.	△ Parz.	—	AAX non supportato (future)
Studio One	6	△ Parz.	△ Parz.	—	VST3, test in corso

[NOTE] Logic Pro richiede il formato AU e presenta limitazioni sul numero di parametri esposti via OSC. Ableton Live rimane la piattaforma di test primaria.

— TEST DRIVER AUDIO

DRIVER	PIATTAFORMA	STATO	NOTE
CoreAudio	macOS	✓ Stabile	Latency < 5ms @ 512 samples
ASIO	Windows	✓ Stabile	Testato con Focusrite, RME, ASIO4ALL
WASAPI	Windows	△ In verifica	Shared mode latency variabile
ALSA/JACK	Linux	△ In verifica	Richiede configurazione utente

— REGRESSION TEST SUITE

Eseguita automaticamente a ogni release candidate:

1. Test di caricamento in tutte le DAW supportate
2. Test di persistenza parametri (salva/carica progetto)
3. Test di automazione OSC (1000 messaggi in 10 secondi)
4. Test di memoria (30 minuti di utilizzo continuo, profiler)
5. Test di crash recovery (kill forzato, restart, stato restored)
6. Test di compatibilità AI provider (Ollama, Gemini, OpenAI)

— CMAKE PRESETS PER PIATTAFORMA

```
{
  "version": 6,
  "configurePresets": [
    {
      "name": "macos-debug",
      "generator": "Xcode",
      "binaryDir": "${sourceDir}/build/macos-debug",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "WHYCREMISI_ENABLE_UNIT_TESTS": "ON",
        "WHYCREMISI_ENABLE_ASAN": "ON"
      }
    },
    {
      "name": "macos-release",
      "generator": "Xcode",
      "binaryDir": "${sourceDir}/build/macos-release",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release",
        "WHYCREMISI_ENABLE_UNIT_TESTS": "OFF"
      }
    },
    {
      "name": "windows-release",
      "generator": "Visual Studio 17 2022",
      "binaryDir": "${sourceDir}/build/win-release",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release"
      }
    },
    {
      "name": "linux-release",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/linux-release",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release"
      }
    }
  ]
}
```

— CACHING CON CCACHE

```
# Abilita ccache per build C++ più veloci in CI
export CCACHE_DIR=~/.ccache
export CCACHE_MAXSIZE=2G
export CCACHE_COMPRESS=1

cmake -S . -B build \
  -DCMAKE_CXX_COMPILER_LAUNCHER=ccache \
  -DCMAKE_C_COMPILER_LAUNCHER=ccache
```

— ESECUZIONE PARALLELA DEI TEST

```
# Test C++ in parallelo con CTest
ctest --test-dir build -j$(nproc) --output-on-failure

# Test React in modalità watch (dev)
npm run test -- --watch

# Test React in CI con coverage
npm run test -- --coverage --maxWorkers=2
```

— CODE COVERAGE REPORTING

STRUMENTO	TARGET	OUTPUT
gcov/lcov	C++ core	HTML + CodeCov
istanbul (v8)	React components	HTML + CodeCov
CodeCov	Aggregato	Badge + PR comment

La soglia di fallimento è configurata come:

- **C++ core:** < 75% → CI warning, < 60% → CI failure
- **React components:** < 60% → CI warning, < 45% → CI failure

• 8. Benchmarking

— METRICHE MISURATE

BENCHMARK	STRUMENTO	TARGET	ATTUALE
Audio Latency	audio_latency_test	< 10ms @ 512 samples	4.2ms (CoreAudio)
FFT Throughput	fft_benchmark	> 100 FFT/s @ 2048 bins	320 FFT/s
WebSocket Message Latency	ws_latency_test	< 5ms round-trip	1.8ms (localhost)
Memory Usage (idle)	memory_profiler	< 150 MB	92 MB
Memory Usage (full session)	memory_profiler	< 350 MB	210 MB
Plugin Load Time	load_time_test	< 1 second	0.4s (VST3)
AI Response Time (local)	ai_benchmark	< 200ms first token	85ms (llama3.2:3b)

— ESEMPIO: BENCHMARK FFT

```
class FftBenchmark : public UnitTest
{
public:
    FftBenchmark() : UnitTest ("FFT Throughput", "Benchmark") {}

    void runTest() override
    {
        beginTest ("2048-bin FFT throughput");
        {
            AudioBuffer<float> buffer (1, 2048);
            buffer.clear();

            auto start = Time::getMillisecondCounterHiRes();
            int iterations = 0;

            while (Time::getMillisecondCounterHiRes() - start < 1000.0)
            {
                FFT fft (11); // 2^11 = 2048
                fft.performFrequencyOnlyForwardTransformation (
                    buffer.getWritePointer(0));
                iterations++;
            }

            auto elapsed = Time::getMillisecondCounterHiRes() - start;
            auto throughput = iterations / (elapsed / 1000.0);

            expect (throughput > 100.0,
                "FFT throughput below target: " + String (throughput));
            logMessage ("FFT throughput: " + String (throughput) + " FFT/s");
        }
    }
};
```

— PROFILING MEMORIA

Utilizziamo **Valgrind** (Linux/macOS) e **UMDH** (Windows) per il profiling memoria:

```
valgrind --tool=massif \
    --massif-out-file=massif.out \
    ./build/WhyCremisi_Tests

ms_print massif.out > memory_profile.txt
```

• 9. Documentazione dei Test

— TEST PLAN DOCUMENT

Ogni release include un test plan documentato in `docs/testing/TEST_PLAN.md` :

docs/testing/

TEST_PLAN.md # Panoramica e strategia generale

TEST_CASES_CPP.md # Casi test C++ dettagliati

TEST_CASES_REACT.md # Casi test React dettagliati

TEST_CASES_E2E.md # Casi test E2E dettagliati

RELEASE_CHECKLIST.md # Checklist pre-release

BUG_REPORT_TEMPLATE.md # Template per segnalazione bug

ENVIRONMENT_SPEC.md # Specifiche ambienti di test

BENCHMARKS.md # Report benchmark storici

— RELEASE CHECKLIST

WHYCREMISI v1.0 — RELEASE CHECKLIST

PRE-RELEASE

- ☐ Tutti i test C++ passano (ctest --output-on-failure)
- ☐ Tutti i test React passano (npm test -- --coverage)
- ☐ Tutti i test E2E passano (npm run test:e2e)
- ☐ Code coverage $\geq 80\%$ C++, $\geq 70\%$ React
- ☐ ASAN non rileva memory leak
- ☐ Build Release su macOS, Windows, Linux

DAW COMPATIBILITY

- ☐ Ableton Live 11 (VST3 + AU)
- ☐ Ableton Live 12 (VST3 + AU)
- ☐ Reaper 7 (VST3)
- ☐ Logic Pro (AU)
- ☐ Cubase 12/13 (VST3)
- ☐ FL Studio 21 (VST3)

AUDIO DRIVERS

- ☐ CoreAudio (macOS)
- ☐ ASIO (Windows)

Descrizione

[Descrizione chiara e concisa del bug]

Riproduzione

1. Apri DAW [nome versione]

2. Carica WhyCremisi

3. [Passaggi per riprodurre]

4. Vedi errore

Comportamento Atteso

[Cosa dovrebbe succedere]

Comportamento Effettivo

[Cosa succede invece]

Ambiente

- OS: [macOS 14.5 / Windows 11 / Ubuntu 24.04]

- DAW: [Ableton Live 12.1 / Reaper 7.2]

- WhyCremisi: [v1.0.0]

- Driver Audio: [CoreAudio / ASIO / WASAPI]

- AI Provider: [Ollama / Gemini / OpenAI]

Log

[Incolla log pertinenti — vedi ~/Library/Logs/WhyCremisi/]

Screenshot / Video

[Se applicabile]

— SPECIFICHE AMBIENTI DI TEST

AMBIENTE	MACOS	WINDOWS	LINUX
CI (GitHub Actions)	macos-14 (Apple Silicon)	windows-2022	ubuntu-22.04
Dev (locale)	macOS 14.5, M3 Max	Windows 11, i9	Ubuntu 24.04, AMD64
QA (manuale)	macOS 14.5, M1 + Intel	Windows 11, Ryzen 9	Ubuntu 24.04, Intel
DAW test	Ableton 12, Logic Pro	Ableton 12, FL Studio	Reaper 7
Audio Interface	RME Babyface Pro FS	Focusrite Scarlett 18i8	— (built-in)